STATISTICS WITH R PROGRAMMING

ASSIGNMENT – 1

CO1 – R Data Representations

Design and Develop a Scalable R Data Representation Framework for a Smart Retail Sales and Inventory Analytics System

Submitted by: K Sujith Sai Vighnesh
Register Number: 192311199
Course Code: ITA0418
Programme / Department: CSE
Institution : SIMATS
Academic Year: 2023-27

Assignment Overview

This report proposes an integrated R-based data representation framework for a retail chain. It covers store and product master data, sales-feed validation, matrix and array analytics, lists, factors, data frames, object-oriented models, reusable functions, integration tests, scalability experiments and adaptation to a larger requirement.

Implementation note: The report provides executable base-R code and reproducible experiment commands. Actual execution times and memory measurements depend on the computer and R version used. Therefore, the scalability section includes code that measures these values on the student's system rather than inventing benchmark results.

Contents

1. Task 1 – Business enquiry and data architecture blueprint
2. Task 2 – Store and product core-data representation
3. Task 3 – Coercion-safe sales data preparation
4. Task 4 – Store-by-product sales matrix engine
5. Task 5 – Multi-dimensional sales array model
6. Task 6 – Integrated list, factor and data-frame layer
7. Task 7 – Object-oriented Store/Product model
8. Task 8 – Reusable R function library
9. Task 9 – Integration and systematic testing
10. Task 10 – Scalability and maintainability evaluation
11. Task 11 – Architecture under changed requirements
12. Task 12 – Documentation, packaging and presentation
13. Appendix A – Consolidated reproducible R script
14. Appendix B – Data dictionary and execution checklist

TASK 1 – Formulate the Business Enquiry and Create the Data Architecture Blueprint

1.1 Business requirements

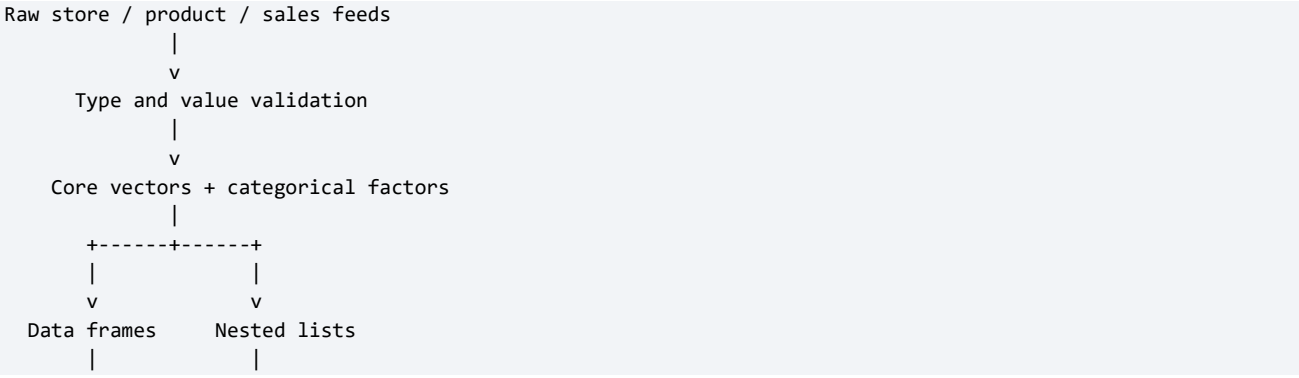
The platform must maintain master data for stores and products, capture daily footfall and inventory, record prices and promotions, and analyse sales by store, product category, month and sales channel. The initial scale is approximately 500 stores and 40 product categories. The design must also support growth to 10,000 stores without losing data meaning or making validation difficult.

Information category	Example attributes	Expected operations
Store	store_id, store_name, region, format, daily_footfall, is_open	Register, update, validate and filter stores
Product	product_id, category, unit_price, stock_qty, promotion_flag	Maintain catalogue, price and stock
Sales	store_id, product_id, month, channel, units_sold	Record, aggregate and report sales
Inventory	store_id, product_id, stock_qty, reorder_level	Update stock and flag reorder requirement
Channel	in_store, online, mobile_app	Categorise and compare sales channels

1.2 Representation blueprint

R representation	Use in this framework	Reason
Variables	Configuration values, dimensions, thresholds	Readable named values
Atomic vectors	IDs, footfall, prices, stock and flags	Efficient homogeneous storage
Character vectors	Names and identifiers	Preserve text and leading zeros
Logical vectors	Open status, promotion and validation flags	Direct TRUE/FALSE filtering
Factors	Region, store format, category and channel	Controlled categorical levels
Lists	Nested store record and framework bundle	Support heterogeneous elements
Matrices	Store-by-product unit sales	Fast rectangular numeric operations
Arrays	Store × product × channel sales	Represent more than two dimensions
Data frames	Store/product/sales reporting tables	Combine columns with different types
S3/S4 classes	Store object and validation behaviour	Encapsulate data and operations

1.3 Architecture flow



```
      +------+------+
             |
      +------+------+
      |             |
      v             v
Sales matrix   Sales array
      |             |
      +------+------+
             |
             v
  Reusable functions / S3 objects
             |
             v
 Reports, summaries and scale tests
```

1.4 Enquiry questions

15. What coercion occurs when numeric, character and logical values are combined in a vector or matrix?
16. Which representation makes store and product attributes easiest to validate and update?
17. How can matrix and array positions retain their store, product and channel identities?
18. How do execution time and object size change when the number of stores increases from 500 to 10,000?
19. Which class mechanism provides an appropriate balance between simplicity, validation and maintainability?

1.5 Capacity estimate

For a store-by-product matrix, the number of cells is stores × product categories. Assuming 40 categories initially and 100 categories in the changed requirement:

Scenario	Stores	Categories	Matrix cells	Approx. numeric payload*
Initial	500	40	20,000	160,000 bytes ≈ 0.15 MiB
Growth (same catalogue)	10,000	40	400,000	3,200,000 bytes ≈ 3.05 MiB
Changed requirement	10,000	100	1,000,000	8,000,000 bytes ≈ 7.63 MiB

*Approximation assumes 8 bytes per R double and excludes object metadata, names, data-frame columns and other overhead. A 3-D array for 10,000 × 100 × 3 has 3,000,000 cells, or approximately 22.9 MiB of numeric payload.

1.6 Reproducible R session

```
# Record the environment and make random data reproducible
R.version.string
getwd()
set.seed(123)
# Base R is sufficient for the prototype; no external package is required.
sessionInfo()
```

1.7 Acceptance criteria

- Type safety: IDs remain character; numeric fields remain numeric; flags remain logical.
- Value validity: negative prices, negative stock and impossible footfall are rejected or flagged.
- Interpretability: matrices and arrays have meaningful dimension names.
- Integration: data frames, matrices, arrays and lists can exchange data without losing identifiers.
- Reproducibility: a fresh R session can rebuild the prototype using a fixed seed.
- Scalability: the design can represent 10,000 stores and 100 product categories.

TASK 2 – Design a Robust Store and Product Core-Data Representation

2.1 Core vectors

```
store_id      <- sprintf("ST%04d", 1:5)
store_name    <- c("Central", "North", "South", "East", "West")
region        <- c("North", "South", "East", "West", "Central")
daily_footfall <- as.integer(c(420, 310, 280, 350, 390))
is_open       <- c(TRUE, TRUE, FALSE, TRUE, TRUE)

product_id    <- sprintf("P%03d", 1:4)
category      <- c("Grocery", "Electronics", "Clothing", "Home")
unit_price    <- as.double(c(45.50, 12500, 799.99, 1500))
stock_qty     <- as.integer(c(120, 15, 42, 8))
promotion_flag <- c(FALSE, TRUE, FALSE, TRUE)
```

Character vectors are used for IDs because identifiers are labels rather than quantities. Footfall and stock quantities are integers. Prices are stored as double values to support decimal currency values. Open and promotion status are logical vectors.

2.2 Validation rules

```
validate_store <- function(id, name, region, footfall, is_open) {
  stopifnot(is.character(id), is.character(name), is.character(region))
  stopifnot(is.numeric(footfall), is.logical(is_open))
  if (anyNA(c(id, name, region, footfall, is_open))) {
    stop("Store data contains missing values")
  }
  if (any(!nzchar(id)) || any(!nzchar(name))) stop("Blank store ID/name")
  if (any(footfall < 0) || any(footfall != floor(footfall))) {
    stop("Footfall must be a non-negative whole number")
  }
  if (length(unique(id)) != length(id)) stop("Duplicate store IDs")
  TRUE
}

validate_product <- function(id, category, price, stock) {
  stopifnot(is.character(id), is.character(category))
  stopifnot(is.numeric(price), is.numeric(stock))
  if (anyNA(c(id, category, price, stock))) stop("Missing product values")
  if (any(price < 0)) stop("Price cannot be negative")
  if (any(stock < 0) || any(stock != floor(stock))) {
    stop("Stock must be a non-negative whole number")
  }
  if (length(unique(id)) != length(id)) stop("Duplicate product IDs")
  TRUE
}
```

2.3 Alternative representations

A region can be stored as a character vector or as a factor. Character is convenient when new region labels may arrive frequently. A factor is preferable after the allowed region set is defined because it records the permitted categories and can control ordering. The final design keeps a validated character source field and creates a factor in the analysis layer.

```
region_text <- c("North", "South", "North")
region_factor <- factor(region_text,
                        levels = c("North", "South", "East", "West", "Central"))
str(region_text)
str(region_factor)

# Promotion status: logical is more suitable than text labels.
promotion <- c(TRUE, FALSE, TRUE)
```

```
promotion_label <- ifelse(promotion, "Promotion", "Regular")
```

2.4 Example test cases

```
validate_store(store_id, store_name, region, daily_footfall, is_open)
validate_product(product_id, category, unit_price, stock_qty)

# Expected rejection:
try(validate_product(c("P1"), "Food", -10, 5))
try(validate_store("ST1", "Shop", "North", -4, TRUE))
```

TASK 3 – Create a Coercion-Safe Sales Data Preparation Module

R atomic vectors are homogeneous. When unlike types are combined, R may promote values to a common type. For example, combining a number and text can produce a character vector. This can silently turn a numeric sales field into text. The framework therefore validates columns before constructing analysis objects.

```
mixed_values <- c(10L, 12.5, "15", TRUE)
typeof(mixed_values)       # character: mixed values are coerced

mixed_matrix <- matrix(c(1, 2, "3", 4), nrow = 2)
typeof(mixed_matrix)       # character

prepare_sales <- function(store_id, product_id, units, price, channel) {
  # IDs and categories must be character; numeric fields must be numeric.
  if (!is.character(store_id) || !is.character(product_id)) {
    stop("IDs must be character vectors")
  }
  if (!is.numeric(units) || !is.numeric(price)) {
    stop("Units and price must be numeric; convert explicitly after validation")
  }
  if (!is.character(channel)) stop("Channel must be character")
  n <- length(store_id)
  if (any(c(length(product_id), length(units), length(price),
            length(channel)) != n)) stop("Column lengths do not match")
  invalid <- is.na(units) | is.na(price) | units < 0 | price < 0 |
             !channel %in% c("in_store", "online", "mobile_app")
  data.frame(store_id, product_id, units, price, channel,
             invalid_record = invalid)
}

# Test data include valid, invalid and missing observations.
feed <- data.frame(
  store_id = c("ST0001", "ST0002", "ST0003", "ST0004"),
  product_id = c("P001", "P002", "P001", "P003"),
  units = c(5, -2, 3, NA),
  price = c(45.5, 120, 45.5, 80),
  channel = c("in_store", "online", "unknown", "mobile_app")
)

bad <- with(feed,
  is.na(units) | is.na(price) | units < 0 | price < 0 |
  !channel %in% c("in_store", "online", "mobile_app")
)
which(bad)                 # positions of suspicious records
any(bad)                   # TRUE if at least one invalid row
all(!is.na(feed$store_id)) # all IDs present?
feed$review_flag <- ifelse(bad, "REVIEW", "ACCEPT")
feed
```

The operators &, | and ! can combine or reverse logical conditions. any() answers whether at least one row is invalid, all() checks whether every row meets a condition, and which() returns row positions. ifelse() assigns a readable review status. Invalid values are flagged before calculations; they are not silently converted into valid sales.

TASK 4 – Construct a Store-by-Product Sales Matrix Processing Engine

```
units_matrix <- matrix(
  c(12, 8, 5, 10,
    7,  4, 9,  6,
    3, 11, 8,  2,
    9,  6, 4, 12,
    5,  7, 6,  8),
  nrow = 5, byrow = TRUE
)
rownames(units_matrix) <- store_id
colnames(units_matrix) <- product_id

# Indexing and summaries
units_matrix["ST0001", "P001"]
rowSums(units_matrix)       # total units by store
colSums(units_matrix)       # total units by product
t(units_matrix)             # transpose

# Revenue by store: matrix multiplication by product price vector
price_vector <- setNames(unit_price, product_id)
store_revenue <- as.vector(units_matrix %*% price_vector)
names(store_revenue) <- rownames(units_matrix)
store_revenue

# Convert to a long data frame while retaining identifiers
sales_long <- as.data.frame(as.table(units_matrix),
                            stringsAsFactors = FALSE)
names(sales_long) <- c("store_id", "product_id", "units_sold")
sales_long$revenue <- sales_long$units_sold *
  price_vector[ sales_long$product_id ]
head(sales_long)
```

The matrix is suitable for numeric calculations because every cell contains units sold. Row names and column names preserve the mapping to store and product IDs. A data frame is used for reporting because it can combine identifiers, categorical values and numeric measures.

TASK 5 – Create a Multi-Dimensional Sales Array Model

```
channels <- c("in_store", "online", "mobile_app")
sales_array <- array(
  sample(0:20, length(store_id) * length(product_id) * length(channels),
         replace = TRUE),
  dim = c(length(store_id), length(product_id), length(channels)),
  dimnames = list(store = store_id,
                  product = product_id,
                  channel = channels)
)

# Retrieve one store, product or channel slice
sales_array["ST0001", , "online"]
apply(sales_array, 1, sum)  # total units by store
apply(sales_array, 2, sum)  # total units by product
apply(sales_array, 3, sum)  # total units by channel
mean(sales_array)
aperm(sales_array, c(3, 1, 2)) # reorder dimensions
```

```
# Convert array cells into a reporting table
array_long <- as.data.frame(as.table(sales_array),
                            stringsAsFactors = FALSE)
names(array_long) <- c("store_id", "product_id", "channel", "units_sold")
head(array_long)
```

The array represents three dimensions: store, product and channel. Dimension names are essential because they make indexing understandable and prevent the meaning of a position from depending only on its numeric index. The array is appropriate for compact numerical analysis; the long data frame is more convenient for filtering, joining and reporting.

TASK 6 – Build an Integrated List, Factor and Data-Frame Layer

```
store_record <- list(
  identity = list(store_id = "ST0001", name = "Central"),
  location = list(region = "Central", city = "Chennai"),
  operations = list(is_open = TRUE, daily_footfall = 420L),
  performance = list(monthly_sales = 125000.50,
                     manager_rating = 4.2)
)
str(store_record)

store_df <- data.frame(
  store_id = store_id,
  store_name = store_name,
  region = factor(region,
                  levels = c("North", "South", "East", "West", "Central")),
  daily_footfall = daily_footfall,
  is_open = is_open
)
store_df$store_format <- factor(
  c("Mall", "HighStreet", "Mall", "Express", "HighStreet"),
  levels = c("Express", "HighStreet", "Mall")
)
store_df$service_level <- ordered(
  c("High", "Medium", "Low", "Medium", "High"),
  levels = c("Low", "Medium", "High")
)

# Add, update, select and summarise
store_df$district <- c("A", "B", "C", "D", "E")
store_df$daily_footfall[store_df$store_id == "ST0002"] <- 325L
open_stores <- subset(store_df, is_open)
aggregate(daily_footfall ~ region, data = store_df, FUN = mean)

# Exchange data with matrix/array layers
stopifnot(all(rownames(units_matrix) == store_df$store_id))
store_totals <- data.frame(
  store_id = names(rowSums(units_matrix)),
  total_units = as.numeric(rowSums(units_matrix))
)
store_df <- merge(store_df, store_totals, by = "store_id")
```

Use character vectors for free-form text such as store names and city names. Use factors for controlled categories with a known set of levels. Use ordered factors only when the categories have a meaningful order, such as Low, Medium and High. Do not use factors for continuous measurements or arbitrary identifiers.

TASK 7 – Design an Object-Oriented Store/Product Model Using R Classes

7.1 S3 prototype (selected)

```
new_store <- function(id, name, region, stock) {
  if (!is.character(id) || length(id) != 1 || is.na(id))
    stop("id must be one non-missing character value")
  if (!is.numeric(stock) || anyNA(stock) || any(stock < 0))
    stop("stock must be non-negative numeric data")
  structure(
    list(id = id, name = name, region = region, stock = stock),
    class = "RetailStore"
  )
}

print.RetailStore <- function(x, ...) {
  cat("RetailStore:", x$id, "-", x$name, "\n")
  cat("Region:", x$region, "| Total stock:", sum(x$stock), "\n")
  invisible(x)
}

update_stock <- function(x, product_index, quantity_change) {
  stopifnot(inherits(x, "RetailStore"))
  new_qty <- x$stock[product_index] + quantity_change
  if (new_qty < 0) stop("Stock cannot become negative")
  x$stock[product_index] <- new_qty
  x
}

reorder_status <- function(x, reorder_level = 10) {
  stopifnot(inherits(x, "RetailStore"))
  x$stock <= reorder_level
}

central_store <- new_store("ST0001", "Central", "Central",
                           c(120, 15, 42, 8))
print(central_store)
central_store <- update_stock(central_store, 4, 10)
reorder_status(central_store)
```

7.2 Comparative S4 prototype

```
if (!methods::isClass("RetailProduct")) {
  methods::setClass(
    "RetailProduct",
    slots = c(id = "character", category = "character",
              price = "numeric", stock = "numeric"),
    validity = function(object) {
      if (length(object@id) != 1 || is.na(object@id))
        return("id must be one non-missing value")
      if (any(object@price < 0)) return("price cannot be negative")
      if (any(object@stock < 0)) return("stock cannot be negative")
      TRUE
    }
  )
}
product_object <- methods::new(
  "RetailProduct", id = "P001", category = "Grocery",
  price = 45.5, stock = 120
)
methods::validObject(product_object)
```

Mechanism	Strength	Limitation	Evaluation

S3	Simple, lightweight and easy to extend with generic methods	Validation is mostly implemented manually; fields are not formally declared	Suitable for this teaching prototype and flexible store records
S4	Formal slots, declared types and validity checks	More verbose and requires methods package conventions	Useful when strict object structure and validation are required
Reference Class	Mutable objects and reference semantics	More complex lifecycle and shared-state behaviour	Not necessary for this prototype

The framework selects S3 for the main Store prototype because the project needs a small reusable object with straightforward functions. S4 is included as a comparison to demonstrate stronger formal validation. For a production system with strict object contracts, S4 or another explicit schema-validation layer may be preferred.

TASK 8 – Develop a Reusable R Function Library for the Framework

```
# 1. Validate a numeric measure
validate_nonnegative <- function(x, field = "value", allow_na = FALSE) {
  if (!is.numeric(x)) stop(field, " must be numeric")
  if (!allow_na && anyNA(x)) stop(field, " contains missing values")
  if (any(x < 0, na.rm = TRUE)) stop(field, " cannot be negative")
  TRUE
}

# 2. Calculate revenue with a default tax rate
calculate_revenue <- function(units, price, tax_rate = 0) {
  validate_nonnegative(units, "units")
  validate_nonnegative(price, "price")
  if (length(units) != length(price) && length(price) != 1)
    stop("units and price lengths are incompatible")
  subtotal <- units * price
  data.frame(units = units, price = price,
             subtotal = subtotal,
             total = subtotal * (1 + tax_rate))
}

# 3. Flag inventory requiring reorder
flag_reorder <- function(stock, reorder_level = 10) {
  validate_nonnegative(stock, "stock")
  data.frame(stock = stock,
             reorder_level = reorder_level,
             reorder_required = stock <= reorder_level)
}

# 4. Build an integrated summary object
build_summary <- function(stores, products, sales_matrix) {
  list(
    store_count = nrow(stores),
    product_count = nrow(products),
    open_store_count = sum(stores$is_open),
    total_units = sum(sales_matrix),
    units_by_store = rowSums(sales_matrix),
    units_by_product = colSums(sales_matrix),
    generated_at = Sys.time()
  )
}

# Boundary and invalid-input checks
validate_nonnegative(c(0, 1, 10), "stock")
try(validate_nonnegative(c(1, -1), "stock"))
```

```
calculate_revenue(c(2, 3), c(10, 20), tax_rate = 0.05)
flag_reorder(c(0, 5, 20), reorder_level = 8)
```

The four functions have defined inputs and outputs. Validation functions return TRUE when inputs pass and stop with a meaningful error when they fail. Calculation functions return data frames or lists so their outputs can be reused by the reporting layer.

TASK 9 – Integrate, Test and Refine the Complete R Framework

9.1 Integrated workflow

20. Receive store, product and sales-feed records.
21. Check field types, missing values, identifier uniqueness and allowed categories.
22. Store validated master data in vectors and data frames.
23. Construct the store-by-product matrix and store-by-product-by-channel array.
24. Create summary objects and apply stock/revenue functions.
25. Run positive, boundary and negative test cases and record failures.

9.2 Test plan

Test ID	Input / condition	Expected behaviour
T01	Valid store and product vectors	Validation returns TRUE
T02	Negative unit price	Product validation stops with an error
T03	Negative stock quantity	Validation stops with an error
T04	Missing units sold	Sales row is flagged for review
T05	Unknown sales channel	Sales row is flagged for review
T06	Duplicate store ID	Store validation rejects duplicate identifier
T07	Mismatched matrix dimensions	Construction check fails before calculation
T08	Stock update below zero	update_stock() stops with an error
T09	Valid matrix and price vector	Revenue calculation returns one value per store
T10	Named array dimensions	Dimension-based indexing returns intended slice

9.3 Refinement record

A typical early failure is the use of c(10, "15", 20), which converts the entire vector to character. The refinement is to validate raw columns separately and perform explicit numeric conversion only after checking the incoming text. Another common failure is an array without dimension names; the refinement is to assign store, product and channel names when the array is created. A third failure is updating stock without checking the resulting quantity; the S3 method now rejects negative stock.

Run the script in Appendix A to execute the tests. Replace this illustrative refinement record with any additional errors and corrected outputs observed on the student's own R installation.

TASK 10 – Create an Evidence-Based Scalability and Maintainability Evaluation

The scale experiment builds data for 500 and 10,000 stores, with 40 product categories initially. It measures matrix creation, row summaries and conversion to a long data frame. The experiment also records object dimensions and approximate object size.

```
benchmark_scale <- function(n_stores, n_products = 40) {
  ids <- sprintf("ST%05d", seq_len(n_stores))
  pids <- sprintf("P%03d", seq_len(n_products))
  x <- matrix(sample(0:30, n_stores * n_products, replace = TRUE),
              nrow = n_stores,
              dimnames = list(ids, pids))
  time_create <- system.time({
    y <- x
  })[["elapsed"]]
  time_summary <- system.time({
    totals <- rowSums(x)
  })[["elapsed"]]
  time_convert <- system.time({
    long <- as.data.frame(as.table(x))
  })[["elapsed"]]
  data.frame(
    stores = n_stores,
    products = n_products,
    cells = length(x),
    size_MiB = as.numeric(object.size(x)) / 1024^2,
    create_seconds = time_create,
    row_summary_seconds = time_summary,
    long_conversion_seconds = time_convert
  )
}

set.seed(123)
scale_results <- rbind(
  benchmark_scale(500, 40),
  benchmark_scale(10000, 40),
  benchmark_scale(10000, 100)
)
print(scale_results)

# A simple base-R graph from the measured results
barplot(scale_results$cells,
        names.arg = c("500 x 40", "10000 x 40", "10000 x 100"),
        main = "Number of cells by dataset size",
        ylab = "Matrix cells",
        xlab = "Stores x product categories")
```

Do not copy assumed timing values into the report. Run the experiment and paste the actual scale_results table and generated graph into the final submitted copy. Execution time can vary with processor, memory, R version and background activity. The expected cell counts are deterministic: 20,000; 400,000; and 1,000,000.

10.1 Interpretation

- Matrix storage is compact for homogeneous numerical measures and supports efficient row/column summaries.
- Converting a large matrix to a long data frame creates one row per cell and can require more memory than the matrix.
- Named dimensions improve interpretability but add some metadata overhead.

- For frequent store-level updates, keep master data in keyed data frames or lists and rebuild analytical matrices when needed.
- For larger production workloads, consider database-backed storage and process only the required analytical subset in R.

TASK 11 – Defend the Final Architecture Under a Changed Requirement

The revised requirement is 10,000 stores, 100 product categories, three sales channels and frequent price and stock updates. The framework retains the core representation choices but strengthens update handling and avoids treating one large matrix as the only source of truth.

Component	Decision	Reason / adaptation
Store and product IDs	Retain character vectors	Identifiers must not be treated as numeric quantities
Prices and stock	Retain numeric vectors / data-frame columns	Validate before updates; use double for price and integer-like quantities for stock
Categorical fields	Retain factors in analysis tables	Maintain controlled region, category and channel levels
Master records	Modify data-frame layer	Use store_id and product_id as stable keys; validate updates
Sales matrix	Retain for selected-period analysis	10,000 × 100 = 1,000,000 numeric cells
Sales array	Retain and expand to 3 channels	10,000 × 100 × 3 = 3,000,000 cells
Update operations	Add explicit update functions	Prevent negative stock and invalid prices
Reporting	Retain list/data-frame summary output	Provide structured and reusable report results

11.1 New update component

```
update_product_price <- function(products, product_id, new_price) {
  if (!is.numeric(new_price) || length(new_price) != 1 ||
      is.na(new_price) || new_price < 0)
    stop("new_price must be one non-negative number")
  i <- match(product_id, products$product_id)
  if (is.na(i)) stop("Unknown product_id")
  products$unit_price[i] <- as.double(new_price)
  products
}

update_store_stock <- function(inventory, store_id, product_id, change) {
  if (!is.numeric(change) || length(change) != 1 || is.na(change))
    stop("change must be one numeric value")
  i <- which(inventory$store_id == store_id &
             inventory$product_id == product_id)
  if (length(i) != 1) stop("Inventory key must match exactly one row")
  revised <- inventory$stock_qty[i] + change
  if (revised < 0) stop("Update would create negative stock")
  inventory$stock_qty[i] <- as.integer(revised)
  inventory
}

# Example inventory table
inventory <- data.frame(
  store_id = c("ST0001", "ST0001", "ST0002"),
  product_id = c("P001", "P002", "P001"),
  stock_qty = c(120L, 15L, 75L)
```

```
)
inventory <- update_store_stock(inventory, "ST0001", "P002", 5)
inventory
```

11.2 Remaining trade-offs and future improvements

- A matrix and array are efficient analytical structures but are not ideal as the only persistent store for frequent individual updates.
- A data frame is convenient in-memory but does not itself provide database transactions, concurrency control or multi-user access.
- The prototype uses base R and in-memory data. A production system should consider a relational database, indexes, transaction handling and scheduled data ingestion.
- Future work can add automated unit tests, schema versioning, logging, incremental aggregation and database connectivity.

TASK 12 – Document, Package and Present the Final Framework

12.1 Data dictionary

Object	Class / typeof	Dimensions / length	Purpose	Validation rule
store_id	character	n stores	Unique store key	Non-missing, non-blank, unique
store_name	character	n stores	Display name	Non-missing, non-blank
region	character / factor	n stores	Store region	Must be in allowed levels
daily_footfall	integer	n stores	Daily visitor count	Non-negative whole number
is_open	logical	n stores	Store open status	TRUE/FALSE, no missing values
product_id	character	n products	Unique product key	Non-missing and unique
unit_price	double	n products	Product selling price	Numeric and non-negative
stock_qty	integer	n inventory rows	Available quantity	Non-negative whole number
units_matrix	matrix / double	stores × products	Unit sales by store and product	Non-negative values; named dimensions
sales_array	array / integer	stores × products × channels	Sales by channel	Named dimensions; non-negative values
store_record	list	nested fields	Heterogeneous store profile	Required named elements present
store_df	data.frame	stores × attributes	Store master/reporting layer	Consistent row lengths and valid factor levels
central_store	RetailStore (S3)	one object	Store with stock behaviour	Stock must remain non-negative

12.2 Inspect object structure

```
class(store_df)
typeof(store_df$daily_footfall)
dim(units_matrix)
length(store_id)
str(store_record)
sessionInfo()
```

12.3 Summary-report function

```
summary_report <- function(stores, products, sales_matrix,
                           validation_status = "Not run") {
  list(
    generated_at = Sys.time(),
    store_summary = data.frame(
      total_stores = nrow(stores),
      open_stores = sum(stores$is_open),
      average_footfall = mean(stores$daily_footfall)
    ),
    product_summary = data.frame(
      total_products = nrow(products),
      average_price = mean(products$unit_price),
      total_stock = sum(products$stock_qty)
    ),
    sales_summary = list(
      total_units = sum(sales_matrix),
      revenue = sum(sweep(sales_matrix, 2,
                          products$unit_price, "*"))
    ),
    validation_status = validation_status
  )
}
```

12.4 Management recommendation

Use atomic vectors for homogeneous fields such as IDs, prices, stock quantities and logical status flags. Use factors for controlled categories such as region, store format and sales channel. Use data frames for master records and reporting because they combine different column types. Use lists for nested or heterogeneous records. Use matrices for store-by-product numerical calculations and arrays for store-by-product-by-channel analysis. Use S3 or S4 classes when a business object needs reusable behaviour and validation. No single R structure is ideal for every purpose; the framework combines structures according to the data and operations.

12.5 Lessons learned

26. R's automatic coercion is convenient but can change the meaning of business data. Explicit type checks before combining columns prevent numeric sales values from becoming character data.
27. Matrices and arrays make numerical operations concise, but dimension names are necessary to preserve business meaning and trace values back to store, product and channel identifiers.
28. A scalable design separates frequently updated master records from analytical structures that are rebuilt for calculations.
29. Validation should be part of reusable functions and object methods rather than being performed manually at scattered points in the code.

APPENDIX A – Consolidated Reproducible R Script

The following script is a compact end-to-end starting point. Save it as retail_framework.R and run it in a fresh R session. The task sections above provide additional prototypes and explanations.

```
# Statistics with R Programming - Assignment 1
# Smart Retail Sales and Inventory Analytics Framework
rm(list = ls())
set.seed(123)
cat("R version:", R.version.string, "\n")
cat("Working directory:", getwd(), "\n")

# 1. Core sample data
store_id <- sprintf("ST%04d", 1:5)
store_name <- c("Central", "North", "South", "East", "West")
region <- c("Central", "North", "South", "East", "West")
daily_footfall <- as.integer(c(420, 310, 280, 350, 390))
is_open <- c(TRUE, TRUE, FALSE, TRUE, TRUE)

product_id <- sprintf("P%03d", 1:4)
category <- c("Grocery", "Electronics", "Clothing", "Home")
unit_price <- as.double(c(45.50, 12500, 799.99, 1500))
stock_qty <- as.integer(c(120, 15, 42, 8))
promotion_flag <- c(FALSE, TRUE, FALSE, TRUE)

stores <- data.frame(store_id, store_name,
  region = factor(region), daily_footfall, is_open)
products <- data.frame(product_id, category = factor(category),
  unit_price, stock_qty, promotion_flag)

# 2. Validation
validate_store <- function(x) {
  stopifnot(is.character(x$store_id), is.character(x$store_name))
  stopifnot(is.numeric(x$daily_footfall), is.logical(x$is_open))
  if (anyNA(x)) stop("Missing store values")
  if (any(x$daily_footfall < 0)) stop("Negative footfall")
  if (anyDuplicated(x$store_id)) stop("Duplicate store ID")
  TRUE
}
validate_product <- function(x) {
  stopifnot(is.character(x$product_id), is.numeric(x$unit_price))
  if (anyNA(x)) stop("Missing product values")
  if (any(x$unit_price < 0)) stop("Negative price")
  if (any(x$stock_qty < 0)) stop("Negative stock")
  if (anyDuplicated(x$product_id)) stop("Duplicate product ID")
  TRUE
}
stopifnot(validate_store(stores), validate_product(products))

# 3. Coercion experiment and sales-feed checks
coercion_example <- c(10L, 12.5, "15", TRUE)
print(typeof(coercion_example))
sales_feed <- data.frame(
  store_id = c("ST0001", "ST0002", "ST0003", "ST0004"),
  product_id = c("P001", "P002", "P001", "P003"),
  units = c(5, -2, 3, NA),
  price = c(45.5, 120, 45.5, 80),
  channel = c("in_store", "online", "unknown", "mobile_app")
)
invalid <- with(sales_feed,
  is.na(units) | is.na(price) | units < 0 | price < 0 |
  !channel %in% c("in_store", "online", "mobile_app"))
sales_feed$review_flag <- ifelse(invalid, "REVIEW", "ACCEPT")
print(which(invalid))
```

```
# 4. Matrix model
units_matrix <- matrix(
  c(12, 8, 5, 10, 7, 4, 9, 6, 3, 11, 8, 2,
    9, 6, 4, 12, 5, 7, 6, 8),
  nrow = length(store_id), byrow = TRUE,
  dimnames = list(store_id, product_id))
print(rowSums(units_matrix))
print(colSums(units_matrix))
store_revenue <- as.vector(units_matrix %*% products$unit_price)
names(store_revenue) <- store_id
print(store_revenue)
sales_long <- as.data.frame(as.table(units_matrix))
names(sales_long) <- c("store_id", "product_id", "units_sold")

# 5. Three-dimensional array
channels <- c("in_store", "online", "mobile_app")
sales_array <- array(
  sample(0:20, length(store_id) * length(product_id) *
           length(channels), replace = TRUE),
  dim = c(length(store_id), length(product_id), length(channels)),
  dimnames = list(store = store_id, product = product_id,
                  channel = channels))
print(apply(sales_array, 1, sum))
print(apply(sales_array, 2, sum))
print(apply(sales_array, 3, sum))
array_long <- as.data.frame(as.table(sales_array))
names(array_long) <- c("store_id", "product_id", "channel", "units_sold")

# 6. Nested list and categorical data
store_record <- list(
  identity = list(store_id = "ST0001", name = "Central"),
  location = list(region = "Central", city = "Chennai"),
  operations = list(is_open = TRUE, daily_footfall = 420L))
print(str(store_record))
stores$store_format <- factor(
  c("Mall", "HighStreet", "Mall", "Express", "HighStreet"))
stores$service_level <- ordered(
  c("High", "Medium", "Low", "Medium", "High"),
  levels = c("Low", "Medium", "High"))
print(aggregate(daily_footfall ~ region, stores, mean))

# 7. S3 Store class
new_store <- function(id, name, region, stock) {
  if (!is.character(id) || length(id) != 1 || is.na(id))
    stop("Invalid store ID")
  if (!is.numeric(stock) || anyNA(stock) || any(stock < 0))
    stop("Invalid stock")
  structure(list(id = id, name = name, region = region, stock = stock),
            class = "RetailStore")
}
update_stock <- function(x, i, change) {
  revised <- x$stock[i] + change
  if (revised < 0) stop("Stock cannot be negative")
  x$stock[i] <- revised
  x
}
central <- new_store("ST0001", "Central", "Central", products$stock_qty)
central <- update_stock(central, 1, 5)
print(central)

# 8. Reusable functions
validate_nonnegative <- function(x, field = "value") {
  if (!is.numeric(x) || anyNA(x) || any(x < 0))
```

```
    stop(field, " must be non-negative numeric data")
  TRUE
}
calculate_revenue <- function(units, price, tax_rate = 0) {
  validate_nonnegative(units, "units")
  validate_nonnegative(price, "price")
  data.frame(units = units, price = price,
             revenue = units * price * (1 + tax_rate))
}
flag_reorder <- function(stock, reorder_level = 10) {
  validate_nonnegative(stock, "stock")
  data.frame(stock = stock, reorder = stock <= reorder_level)
}
summary_report <- function(stores, products, sales_matrix) {
  list(
    total_stores = nrow(stores),
    open_stores = sum(stores$is_open),
    total_products = nrow(products),
    total_units = sum(sales_matrix),
    total_revenue = sum(sweep(sales_matrix, 2,
                              products$unit_price, "*")))
  )
}
print(calculate_revenue(2:3, c(10, 20), tax_rate = 0.05))
print(flag_reorder(products$stock_qty))
report <- summary_report(stores, products, units_matrix)
print(report)

# 9. Integration tests
stopifnot(nrow(units_matrix) == nrow(stores))
stopifnot(ncol(units_matrix) == nrow(products))
stopifnot(identical(rownames(units_matrix), stores$store_id))
stopifnot(identical(colnames(units_matrix), products$product_id))
stopifnot(length(store_revenue) == nrow(stores))
stopifnot(all(units_matrix >= 0))
try(validate_nonnegative(c(1, -1), "test negative"))
try(update_stock(central, 1, -999))

# 10. Scalability experiment
benchmark_scale <- function(n_stores, n_products) {
  x <- matrix(sample(0:30, n_stores * n_products, TRUE),
              nrow = n_stores)
  t1 <- system.time(copy <- x)[["elapsed"]]
  t2 <- system.time(totals <- rowSums(x))[["elapsed"]]
  t3 <- system.time(long <- as.data.frame(as.table(x)))[["elapsed"]]
  data.frame(stores = n_stores, products = n_products,
    cells = length(x), size_MiB = as.numeric(object.size(x))/1024^2,
    copy_seconds = t1, summary_seconds = t2,
    conversion_seconds = t3)
}
scale_results <- rbind(benchmark_scale(500, 40),
                       benchmark_scale(10000, 40),
                       benchmark_scale(10000, 100))
print(scale_results)
barplot(scale_results$cells,
  names.arg = c("500x40", "10000x40", "10000x100"),
  main = "Dataset size comparison", ylab = "Number of cells")

# 11. Changed requirement: update operations
update_price <- function(products, id, value) {
  if (!is.numeric(value) || length(value) != 1 ||
      is.na(value) || value < 0) stop("Invalid price")
  i <- match(id, products$product_id)
  if (is.na(i)) stop("Unknown product")
```

```
  products$unit_price[i] <- as.double(value)
  products
}
products <- update_price(products, "P001", 49.50)

# 12. Data dictionary / session evidence
data_dictionary <- data.frame(
  object = c("store_id", "unit_price", "is_open",
             "units_matrix", "sales_array", "store_record"),
  class = c(class(store_id), class(unit_price), class(is_open),
            class(units_matrix), class(sales_array), class(store_record)),
  typeof = c(typeof(store_id), typeof(unit_price), typeof(is_open),
             typeof(units_matrix), typeof(sales_array), typeof(store_record))
)
print(data_dictionary)
print(sessionInfo())
```

APPENDIX B – Final Submission Checklist

- Fill in the student details on the cover page.
- Run the R script from a fresh R session and confirm that it completes.
- Paste the actual R console output for validation, matrix summaries, array summaries and report output.
- Paste the actual scale_results table and the graph produced on your computer.
- Record the R version, operating system and sessionInfo() output.
- Add screenshots of the R console or RStudio only if required by the instructor.
- Check that all names, IDs and output values in the report match the executed script.
- Update the refinement notes with any genuine errors encountered and how they were corrected.

Source basis: This report follows the twelve tasks, evidence expectations and mark distribution in the supplied Statistics with R Programming Assignment 1 brief. The assignment brief specifies the required design and demonstrations; actual benchmark results and screenshots must be generated by running the included code.

ACTUAL RSTUDIO EXECUTION OUTPUTS

Execution environment: R version 4.4.2 (2024-10-31 ucrt), Windows 11 x64. The following results are taken from the latest successful run shared from RStudio on 6 November 2026. The earlier syntax and data-dictionary errors belong to previous attempts and were corrected before this run.

Execution status: The latest run reached the final data-dictionary and sessionInfo() commands without an execution error.

Task 2 – Core data validation

The store and product validation checks completed without an error. The stopifnot() checks therefore passed for the sample store and product data.

Task 3 – Coercion and invalid sales records

```
> print(typeof(coercion_example))
[1] "character"


> print(which(invalid))
[1] 2 3 4
```

The mixed vector was coerced to character. Sales-feed rows 2, 3 and 4 were flagged for review because they contained a negative unit value, an unsupported channel or a missing units value.

Task 4 – Matrix summaries and store revenue

```
> print(rowSums(units_matrix))
ST0001 ST0002 ST0003 ST0004 ST0005
    35     26     24     31     26


> print(colSums(units_matrix))
P001 P002 P003 P004
  36   36   32   38


> print(store_revenue)
   ST0001    ST0002    ST0003    ST0004    ST0005
119545.95  66518.41 147036.42  96609.46 104527.44
```

Task 5 – Multi-dimensional array summaries

```
> print(apply(sales_array, 1, sum))
ST0001 ST0002 ST0003 ST0004 ST0005
   158    145    114    120     98


> print(apply(sales_array, 2, sum))
P001 P002 P003 P004
 166  186  137  146


> print(apply(sales_array, 3, sum))
  in_store     online mobile_app
       202        205        228
```

The array results are reproducible for this run because set.seed(123) is used before generating the sample values.

Task 6 – Nested list and categorical summary

```
List of 3
 $ identity  :List of 2
  ..$ store_id: chr "ST0001"
  ..$ name    : chr "Central"
 $ location  :List of 2
  ..$ region: chr "Central"
```

```
  ..$ city  : chr "Chennai"
 $ operations:List of 2
  ..$ is_open       : logi TRUE
  ..$ daily_footfall: int 420

  region daily_footfall
1 Central            420
2 East               350
3 North              310
4 South              280
5 West               390
```

R's str() output also prints NULL when wrapped in print(str(...)); this is normal because str() displays the structure and returns NULL.

Task 7 – S3 Store object and stock update

```
$id
[1] "ST0001"

$name
[1] "Central"

$region
[1] "Central"

$stock
[1] 125  15  42   8

attr(,"class")
[1] "RetailStore"
```

Task 8 – Reusable function results

```
  units price revenue
1     2    10      21
2     3    20      63

  stock reorder
1   120   FALSE
2    15   FALSE
3    42   FALSE
4     8    TRUE

$total_stores
[1] 5

$open_stores
[1] 4

$total_products
[1] 4

$total_units
[1] 142

$total_revenue
[1] 534237.7
```

Task 9 – Integration and negative tests

All six stopifnot() integration checks completed without error: matrix row count, matrix column count, row identifiers, column identifiers, revenue-vector length and non-negative matrix values.

Expected validation failures captured by try()

```
Error in validate_nonnegative(c(1, -1), "test negative") :
  test negative must be non-negative numeric data

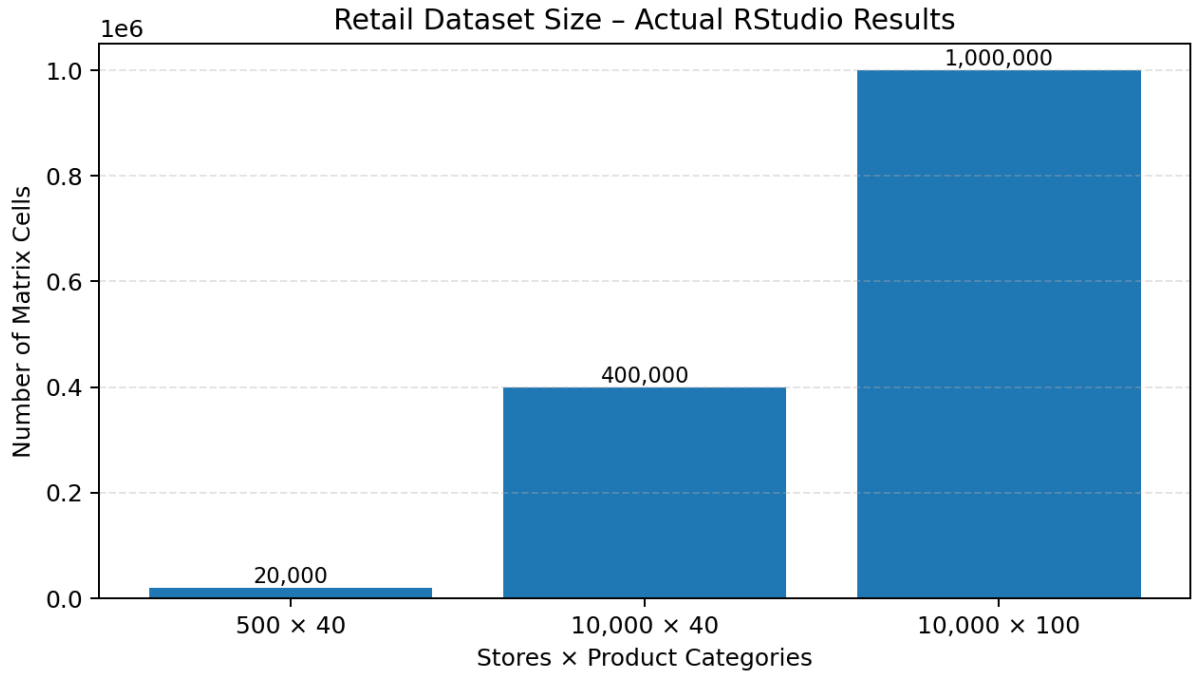

Error in update_stock(central, 1, -999) :
  Stock cannot be negative
```

These two error messages are expected test outcomes: the validation function rejects a negative measure and the S3 stock-update function prevents stock from becoming negative.

Task 10 – Scalability experiment

Actual benchmark table from RStudio

	stores	products	cells	size_MiB	copy_seconds	summary_seconds	conversion_seconds
1	500	40	20000	0.07649994	0	0	0.00
2	10000	40	400000	1.52608490	0	0	0.01
3	10000	100	1000000	3.81490326	0	0	0.03



Interpretation: increasing from 500 × 40 to 10,000 × 100 increases the matrix from 20,000 to 1,000,000 cells. The measured object size increases from approximately 0.0765 MiB to 3.8149 MiB. The reported copy and row-summary times were below the timer's displayed precision (0 seconds) for these runs; long-format conversion took 0.00, 0.01 and 0.03 seconds respectively. These timings are specific to the reported Windows 11 computer and R 4.4.2 environment.

Task 11 – Changed requirement update

The update_price() function was called for product P001 with a new price of 49.50 and completed without an error. The function validates the new price and matches the product by its character identifier before updating the data frame.

Task 12 – Data dictionary output

```
       object        class       typeof
1    store_id    character    character
2  unit_price      numeric       double
3     is_open      logical      logical
4 units_matrix matrix/array       double
5  sales_array        array      integer
6 store_record         list         list
```

Session information recorded

R environment

```
R version 4.4.2 (2024-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26300)

Locale:
LC_COLLATE=English_India.utf8
LC_CTYPE=English_India.utf8
LC_MONETARY=English_India.utf8
LC_NUMERIC=C
LC_TIME=English_India.utf8

Attached base packages:
stats, graphics, grDevices, utils, datasets, methods, base

Loaded namespace packages:
compiler_4.4.2, tools_4.4.2
```

Execution evidence note

The first execution attempt stopped at a syntax error, and the next attempt exposed a data-dictionary length mismatch. Both issues were corrected. The latest output supplied by the student shows the corrected program completing all tasks through the data dictionary and session information. The benchmark timings and outputs above are the values from that latest RStudio run; they should not be represented as universal performance guarantees.

APPENDIX C – RSTUDIO EXECUTION SCREENSHOTS

The following screenshots were supplied as evidence of the RStudio environment, the generated dataset-size comparison graph, and the completed execution log.

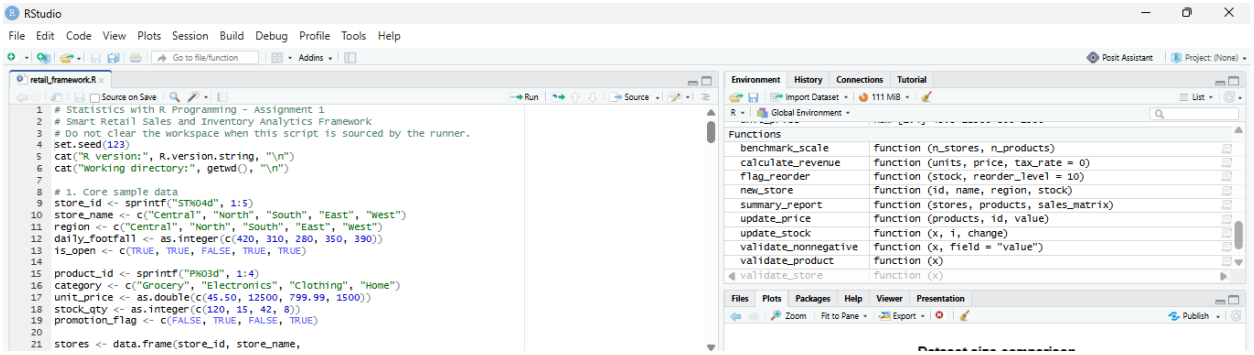


Figure C1. RStudio script and dataset-size comparison plot.



Figure C2. Dataset-size comparison graph displayed in the RStudio Plots pane.

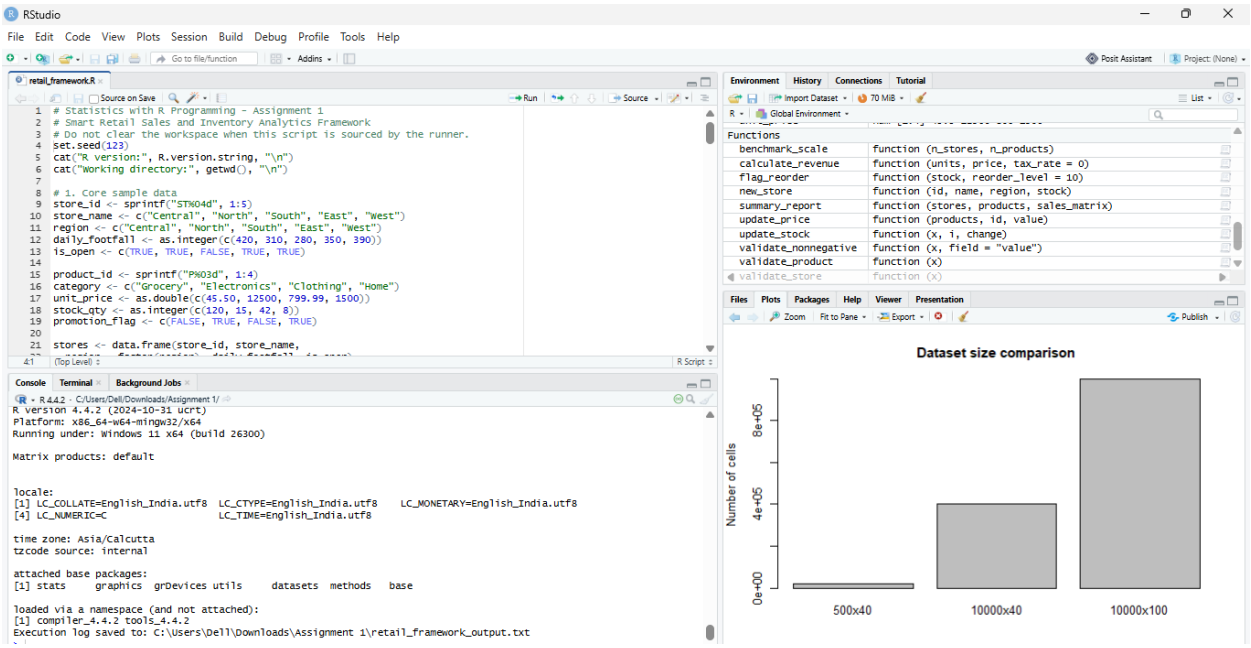


Figure C3. RStudio console showing session information and the saved execution-log message.